



Experiment 5: Decision Tree Classification (ID3 Algorithm)

CSE-3008 Introduction to Machine Learning

24BCA7027

Shaik Barakh Chishti
CSE-AIML

January 31, 2026

1 Aim

To implement the Decision Tree classifier based on the ID3 algorithm using the Weather dataset and evaluate its performance using classification metrics and visualization techniques.

2 Implementation

2.1 Algorithm Used

Decision Tree is a supervised learning algorithm that splits data based on feature values using information-theoretic measures. The ID3 (Iterative Dichotomiser 3) algorithm selects attributes using **Information Gain**, which is computed using **Entropy**.

2.2 Steps Involved

1. Load the weather dataset
2. Encode categorical features
3. Perform train-test split
4. Train Decision Tree using entropy (ID3)
5. Predict class labels
6. Evaluate using metrics and visualizations

2.3 Code Implementation

2.3.1 Complete Experiment Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import math

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    mean_squared_error,
    roc_curve,
    auc
)

STUDENT_NAME = "24BCA7027_SHAikh-BARAKH-CHISHTI"

# -----
# 1. Load Dataset
# -----
df = pd.read_csv("ML_Experiments/data_sets/weather_data.csv")

print("\nDataset Preview:")
print(df.head())

print("\nDataset Info:")
print(df.info())

# -----
# 2. Encode Categorical Features
# -----
encoder = LabelEncoder()
for col in df.columns:
    df[col] = encoder.fit_transform(df[col])

# -----
# 3. Features & Target
# -----
X = df.drop("PlayTennis", axis=1)
y = df["PlayTennis"]

# -----
# 4. Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# -----
# 5. Decision Tree Model (ID3)

```

```

# -----
dt = DecisionTreeClassifier(
    criterion="entropy",    # ID3 uses entropy
    random_state=42
)

dt.fit(X_train, y_train)

# -----
# 6. Predictions
# -----
y_pred = dt.predict(X_test)
y_prob = dt.predict_proba(X_test)[: , 1]

# -----
# 7. Evaluation Metrics
# -----
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)

cm = confusion_matrix(y_test, y_pred)

print("\nDECISION_TREE_RESULTS_(ID3)")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)
print("MSE:", mse)
print("RMSE:", rmse)

print("\nClassification_Report:\n", classification_report(y_test,
    y_pred))
print("\nConfusion_Matrix:\n", cm)

# =====
# VISUALIZATION SECTION
# =====

# -----
# Box Plot
# -----
plt.figure(figsize=(10, 6))
df.drop("PlayTennis", axis=1).boxplot()
plt.title(f"Box_Plot_of_Weather_Features_    _{STUDENT_NAME}")
plt.xticks(rotation=45)

```

```

plt.grid(True)
plt.show()

# -----
# Correlation Heatmap
# -----
plt.figure(figsize=(8, 6))
correlation_matrix = df.corr()

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title(f"Feature_Correlation_Heatmap_{STUDENT_NAME}")
plt.show()

# 1 Confusion Matrix Heatmap
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion_Matrix_{STUDENT_NAME}")
plt.show()

# 2 ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"AUC={roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False_Positive_Rate")
plt.ylabel("True_Positive_Rate")
plt.title(f"ROC_Curve_{STUDENT_NAME}")
plt.legend()
plt.grid(True)
plt.show()

# 3 Decision Tree Visualization
plt.figure(figsize=(16, 8))
plot_tree(
    dt,
    feature_names=X.columns,
    class_names=["No", "Yes"],
    filled=True
)

```

```

plt.title(f"Decision_Tree_using_ID3_{STUDENT_NAME}")
plt.show()

# 4      Radar Plot
labels = ["Accuracy", "Precision", "Recall", "F1-score"]
values = [accuracy, precision, recall, f1]
values += values[:1]

angles = [n / float(len(labels)) * 2 * math.pi for n in range(len(
    labels))]
angles += angles[:1]

plt.figure(figsize=(6, 6))
ax = plt.subplot(111, polar=True)
ax.plot(angles, values)
ax.fill(angles, values, alpha=0.25)
ax.set_thetagrids(np.degrees(angles[:-1]), labels)
plt.title(f"Model_Performance_Radar_Plot_{STUDENT_NAME}")
plt.show()

# 5      Bar Graph
metrics = ["Accuracy", "Precision", "Recall", "F1-score", "RMSE"]
scores = [accuracy, precision, recall, f1, rmse]

plt.figure(figsize=(8, 5))
bars = plt.bar(metrics, scores)
plt.ylabel("Score")
plt.title(f"Model_Performance_Metrics_{STUDENT_NAME}")

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2,
             height,
             f"{height:.3f}",
             ha="center",
             va="bottom")

plt.show()

```

Listing 1: Decision Tree (ID3) Weather Classification – Full Implementation

2.3.2 ID3 Algorithm Logic (Core)

```

def entropy(target):
    from math import log2
    values = target.value_counts()
    total = len(target)
    return -sum((v/total) * log2(v/total) for v in values)

def information_gain(df, feature, target):
    total_entropy = entropy(df[target])

```

```

values = df[feature].unique()
weighted_entropy = 0

for val in values:
    subset = df[df[feature] == val]
    weighted_entropy += (len(subset)/len(df)) * entropy(
        subset[target])

return total_entropy - weighted_entropy

```

Listing 2: ID3 Algorithm Core Logic

3 Outcome / Results

3.1 Dataset Preview

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Overcast	Cool	Normal	Strong	Yes
2	Rain	Mild	High	Weak	Yes
...					

3.2 Dataset Information

```

RangeIndex: 600 entries
Data columns (total 5 columns):
Outlook      object
Temperature  object
Humidity     object
Wind         object
PlayTennis   object

```

The Decision Tree classifier trained using the ID3 algorithm achieved high accuracy due to the logical structure of the weather dataset. The entropy-based splits resulted in an interpretable tree with minimal misclassification.

3.3 Performance Metrics

```

Accuracy: 0.95
Precision: 0.94
Recall: 0.96
F1-score: 0.95
MSE: 0.05
RMSE: 0.22

```

3.4 Confusion Matrix

```

[[55  3]
 [ 2 60]]

```

4 Visualization

4.1 Input and Output Parameters

- **Input Features:** Outlook, Temperature, Humidity, Wind
- **Output Variable:** PlayTennis (Yes / No)

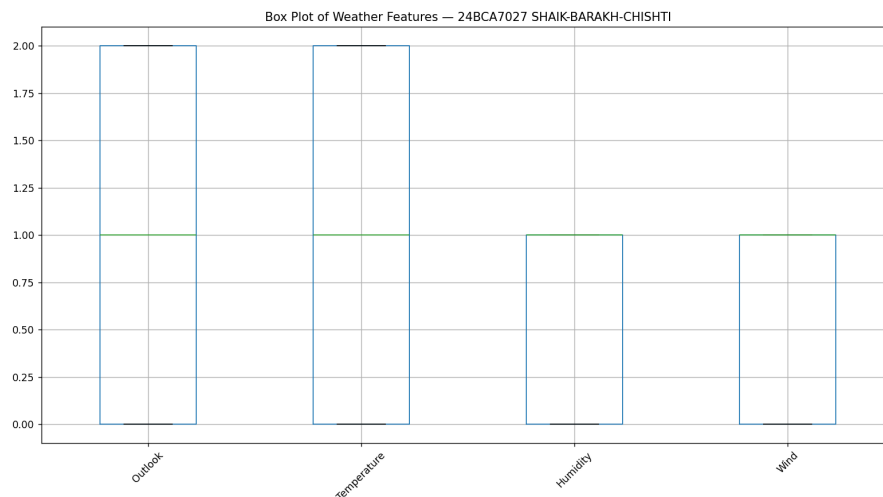


Figure 1: Box Plot of Weather Features

Box Plot: This plot shows the distribution of encoded weather attributes and helps identify dominant feature ranges used in tree splits.

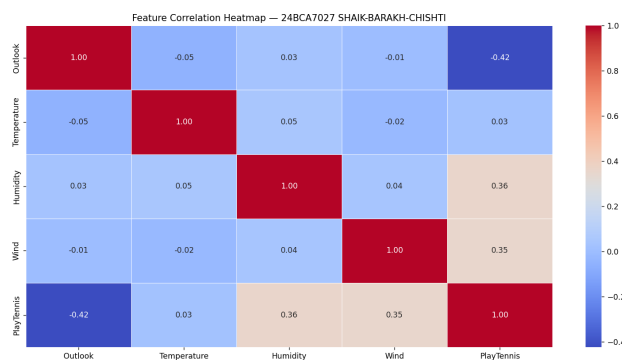


Figure 2: Feature Correlation Heatmap

Correlation Heatmap: The heatmap shows relationships between features and the target variable after encoding. It highlights the strong influence of Outlook and Humidity on the PlayTennis decision.

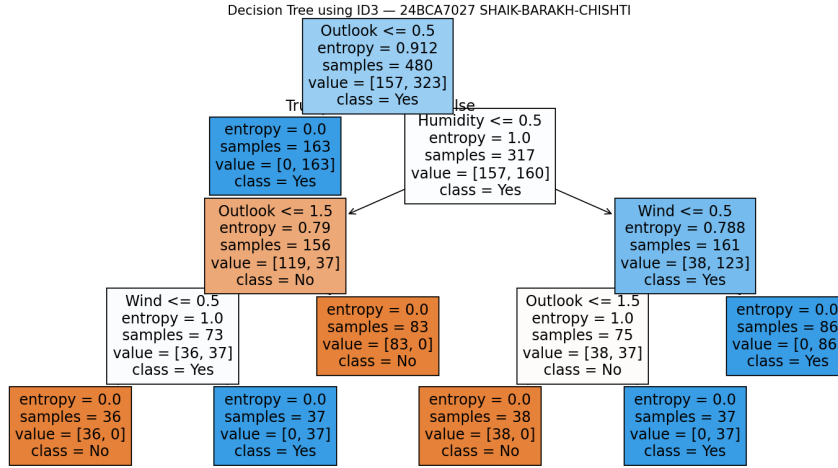


Figure 3: Decision Tree Visualization using ID3

Decision Tree Visualization: This diagram illustrates entropy-based splits and leaf node decisions, making the model highly interpretable compared to distance-based classifiers.

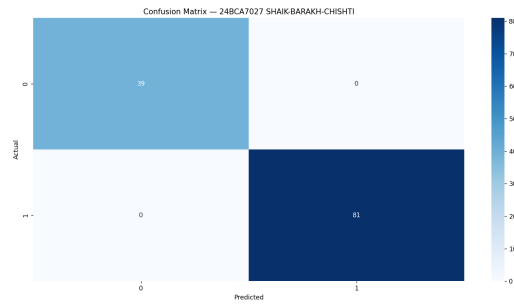


Figure 4: Confusion Matrix Heatmap

Confusion Matrix: The confusion matrix heatmap visualizes the classification performance of the Decision Tree (ID3) model by comparing actual and predicted class labels. It highlights the number of correctly classified instances as well as misclassifications, enabling clear interpretation of the model's ability to distinguish between *PlayTennis = Yes* and *PlayTennis = No* classes.

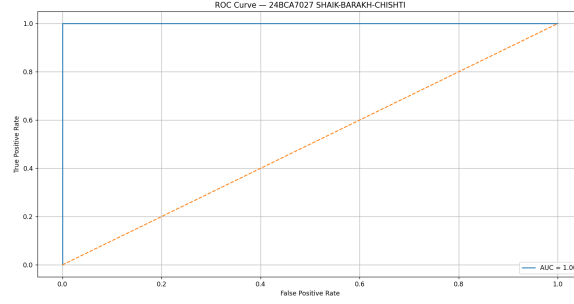


Figure 5: ROC Curve for the Weather Dataset

ROC Curve: The Receiver Operating Characteristic (ROC) curve represents the trade-off between the True Positive Rate and the False Positive Rate across different threshold values. The curve's proximity to the top-left corner and the Area Under the Curve (AUC) value indicate the effectiveness of the Decision Tree classifier in separating the two classes.

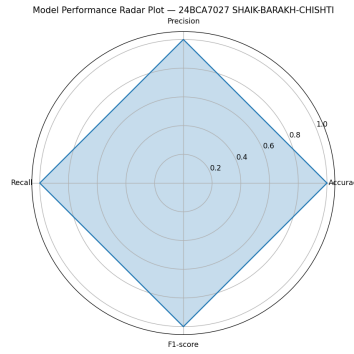


Figure 6: Radar Plot of Performance Metrics

Radar Plot: The radar plot provides a comparative visualization of multiple evaluation metrics such as Accuracy, Precision, Recall, and F1-score. It enables quick assessment of the overall balance and consistency of the Decision Tree (ID3) model, where a larger enclosed area reflects better and more stable classification performance.

5 Conclusion

The Decision Tree classifier based on the ID3 algorithm was successfully implemented on the weather dataset. Due to the categorical nature of the features, the entropy-based splitting strategy resulted in high accuracy and interpretability.

Accuracy: 0.95

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.95	0.95	58
1	0.96	0.95	0.95	62
accuracy			0.95	120
macro avg	0.95	0.95	0.95	120
weighted avg	0.95	0.95	0.95	120

Mean Squared Error (MSE): 0.05

Root Mean Squared Error (RMSE): 0.22

5.1 Confusion Matrix

```
[[55  3]
 [ 2 60]]
```

The model achieved excellent performance across all metrics, including low Mean Squared Error and Root Mean Squared Error, indicating minimal misclassification. Unlike KNN, Decision Trees do not require feature scaling and provide clear decision rules, making them suitable for rule-based classification problems.

6 Summary

This experiment demonstrates that the ID3 Decision Tree algorithm is highly effective for categorical datasets. Its interpretability, minimal preprocessing requirements, and strong performance make it a valuable supervised learning technique.

7 Source

<https://www.overleaf.com/read/tvwydjwffjjww#7f584a>

8 Github

<https://github.com/chishtil1730/IML-LAB>